

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

Facultad de Ciencias Puras | Ingeniería Informática

INF781 — Seguridad de Software

EXAMEN DE PRIMER PARCIAL

Auditoría y Corrección de Vulnerabilidades

Aplicación PHP pura + PostgreSQL

Campo	Detalle
Modalidad	Laboratorio, individual, sin consulta a internet
Duración	90 minutos
Tecnología	PHP 8.3+ puro + PostgreSQL 14+
Puntaje total	100 puntos
Entregables	Código corregido (ZIP) + Informe (PDF)
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán

Gestión 2026 — Potosí, Bolivia

1. Contexto del Examen

Durante las guías 1 a 4 de esta materia, y en las dos prácticas previas, aprendiste a construir aplicaciones con protecciones progresivas (configuración segura, validación, hashing robusto, sesiones seguras, CSRF) y también a construir deliberadamente aplicaciones vulnerables en PHP puro para comprender el origen de cada vulnerabilidad. Este examen cierra ese ciclo: ahora te enfrentas a una aplicación existente con fallas reales, y tu trabajo es actuar como un ingeniero de seguridad.

Recibirás un repositorio semilla llamado examen-insecure-notes, una aplicación PHP pura de gestión de notas personales (CRUD + login) que tiene varias vulnerabilidades de seguridad introducidas intencionalmente. Tu tarea consiste en, para tres secciones específicas: identificar la vulnerabilidad en el código, explotarla desde el navegador capturando evidencia, y corregirla aplicando las técnicas aprendidas durante el semestre.

Advertencia ética

La aplicación entregada es intencionalmente vulnerable y debe ejecutarse únicamente en localhost durante este examen. No está permitido desplegarla, ni reutilizar las técnicas de explotación fuera del contexto educativo evaluado aquí. El uso indebido de estas técnicas contra sistemas sin autorización es ilegal.

1.1 Habilidades que se evalúan

- Leer código PHP existente y localizar vulnerabilidades de inyección y escape.
- Construir un payload de explotación que demuestre el impacto real de la falla.
- Capturar evidencia visual del ataque exitoso (screenshots).
- Aplicar la mitigación correcta (prepared statements, escape de salida, hashing robusto, token CSRF, flags de cookie).
- Comunicar el diagnóstico y la corrección en un informe breve pero preciso.

2. Preparación del Entorno (10 min)

Antes de empezar a resolver las secciones, verifica que la aplicación semilla corre correctamente. Si algo falla, avisa al docente ANTES de que inicie el cronómetro de las secciones evaluables.

Paso 1: Descomprimir el repositorio

```
cd ~/Desktop
unzip examen-insecure-notes.zip
cd examen-insecure-notes
```

Paso 2: Crear la base de datos

```
psql -U postgres -c "DROP DATABASE IF EXISTS examen_insecure_notes;"
psql -U postgres -c "CREATE DATABASE examen_insecure_notes;"
psql -U postgres -d examen_insecure_notes -f sql/schema.sql
psql -U postgres -d examen_insecure_notes -f sql/seed.sql
```

Paso 3: Iniciar el servidor

```
php -S localhost:8000
```

Paso 4: Verificar funcionamiento

Abre <http://localhost:8000> en tu navegador y confirma:

- Puedes iniciar sesión con `admin@uatf.edu` / `admin123`
- Ves 3 notas listadas en la pantalla principal
- Puedes crear, editar y eliminar notas sin errores

Si algo falla en el setup

Levanta la mano ANTES de iniciar la primera sección evaluable. El tiempo del setup no cuenta contra los 90 minutos de las secciones, pero no hay extensiones por problemas de configuración no reportados a tiempo.

Sección 1 — SQL Injection (30 puntos)

TIEMPO SUGERIDO: 20 MIN | PUNTAJE: 30 PTS

Identifica la vulnerabilidad, bypass del login, parche con prepared statements.

1.1 Identificación

Abre el archivo login.php en tu editor. Observa cómo se construye la consulta SQL que autentica al usuario. Responde en tu informe:

- ¿En qué línea(s) del archivo login.php se encuentra la vulnerabilidad de SQL Injection?
- ¿Por qué es vulnerable (qué principio de seguridad se está violando)?
- ¿Qué diferencia hay entre esta construcción y un prepared statement?

1.2 Explotación

Demuestra que la vulnerabilidad es explotable. Debes iniciar sesión como el usuario admin@uatf.edu SIN conocer su contraseña real.

Pista: Recuerda los payloads del Ejercicio 1 de la Práctica 2 (Login Bypass). En este caso, el comparador de PostgreSQL para comentar el resto de la consulta es el mismo que usaste allí.

Una vez dentro, ve a la lista de notas del administrador.

Capturas requeridas (CAP-1.A y CAP-1.B)

- CAP-1.A: El formulario de login con el payload escrito en el campo email (antes de enviar).
- CAP-1.B: La pantalla "Mis Notas" mostrando las 3 notas del administrador, con la URL y el nombre "Administrador" visibles en la cabecera.

1.3 Corrección

Modifica login.php para que use un prepared statement real con PDO y elimine por completo la interpolación de variables en la SQL.

Requisitos del parche: usar PDO con ATTR_EMULATE_PREPARES en false, parametrizar el email mediante placeholders nombrados, y mantener el flujo de redirección a /notes/index.php tras un login exitoso.

Captura requerida (CAP-1.C)

- CAP-1.C: Intentar el mismo payload del punto 1.2 después del parche; la aplicación debe responder "Credenciales inválidas" y NO permitir el ingreso.

Desglose de 30 puntos

Criterio	Pts
Identificación correcta de las líneas vulnerables y explicación técnica	8
Payload de bypass funcional (CAP-1.A + CAP-1.B)	10
Parche con prepared statement PDO correctamente implementado	10
Verificación del parche (CAP-1.C)	2

Sección 2 — XSS + Hashing MD5 (30 puntos)

TIEMPO SUGERIDO: 25 MIN | PUNTAJE: 30 PTS

Cadena de ataque: XSS reflejado en notas + contraseñas en MD5 fácilmente reversibles.

Esta sección evalúa DOS vulnerabilidades relacionadas, porque un atacante realista las combinaría: primero roba la sesión por XSS, luego intercepta el hash MD5 de la BD y lo revierte con una tabla rainbow.

2.1 Parte A — Cross-Site Scripting (XSS)

Identificación

Abre `notes/index.php`. Observa cómo se imprimen los campos `title` y `content` de cada nota.

- ¿En qué líneas se produce la vulnerabilidad XSS?
- ¿Qué función o directiva FALTA para que las salidas sean seguras?

Explotación

Inicia sesión como el administrador (usa el bypass de la Sección 1 si ya la completaste, o las credenciales `admin123` si prefieres). Crea una nueva nota con el siguiente payload en el campo CONTENIDO:

```
<script>alert(document.cookie)</script>
```

Guarda la nota y navega a la lista de notas. Verifica que aparece un alert mostrando el PHPSESSID (la cookie de sesión, que es legible porque `HttpOnly` está desactivado en `config.php`).

Capturas requeridas (CAP-2.A y CAP-2.B)

- CAP-2.A: El formulario "Nueva Nota" con el payload escrito en el campo contenido (antes de guardar).
- CAP-2.B: El `alert()` ejecutándose en la lista de notas mostrando el valor de `document.cookie`.

Corrección

Modifica `notes/index.php` para que todas las salidas dinámicas pasen por `htmlspecialchars()` con las banderas `ENT_QUOTES` | `ENT_HTML5` y codificación UTF-8.

Sugerencia: define un helper `e($value)` en `config.php` y reutilízalo en toda la vista. Incluye también `title`, `user_name` y `created_at`.

Captura requerida (CAP-2.C)

- CAP-2.C: Después del parche, la misma nota con el payload debe mostrarse como texto literal (sin ejecutar el alert).

2.2 Parte B — Hashing MD5

Identificación

En login.php se usa md5() para hashear la contraseña enviada por el usuario. Los hashes almacenados en la tabla users también son MD5. Responde:

- ¿Por qué MD5 es inseguro para almacenar contraseñas en 2026?
- ¿Qué es una rainbow table y por qué funciona contra MD5 sin salt?

Explotación

Conéctate a PostgreSQL y extrae el hash MD5 del usuario admin@uatf.edu:

```
psql -U postgres -d examen_insecure_notes \
-c "SELECT email, password FROM users WHERE email='admin@uatf.edu';"
```

Copia el valor del hash y pégalo en un sitio de crackeo MD5 en línea (por ejemplo, crackstation.net). Reportarás que el hash se revierte en segundos porque admin123 está en todas las wordlists populares.

Captura requerida (CAP-2.D)

- CAP-2.D: Pantalla de crackstation (u otro servicio equivalente) mostrando el hash MD5 del admin revertido al texto plano admin123.

Corrección

Modifica el flujo completo para usar bcrypt (password_hash + password_verify):

- **En login.php:** reemplazar md5(\$password) por una comparación con password_verify(\$password, \$user['password']).
- **En sql/seed.sql:** reemplazar los hashes MD5 por hashes bcrypt generados con password_hash(\$pw, PASSWORD_BCRYPT).
- Re-ejecutar el seeder para que los usuarios de prueba funcionen con las nuevas credenciales.

Captura requerida (CAP-2.E)

- CAP-2.E: Login exitoso con admin@uatf.edu / admin123 después del parche (y tabla users mostrando hashes bcrypt \$2y\$... en lugar de MD5).

Desglose de 30 puntos

Criterio	Pts
XSS: identificación y payload ejecutado (CAP-2.A + CAP-2.B)	8
XSS: parche con htmlspecialchars aplicado a todas las salidas (CAP-2.C)	8
MD5: explicación técnica y hash revertido (CAP-2.D)	6
MD5: migración a bcrypt en login + seeder (CAP-2.E)	8

Sección 3 — CSRF + Sesiones Inseguras (25 puntos)

TIEMPO SUGERIDO: 25 MIN | PUNTAJE: 25 PTS

Eliminar notas de una víctima desde un sitio externo; luego cerrar el vector.

3.1 Identificación

Revisa config.php y todos los formularios de notes/. Responde en tu informe:

- ¿Qué tres flags de cookie de sesión están desactivados en config.php y qué riesgo introduce cada uno?
- ¿Falta session_regenerate_id() en algún punto crítico? ¿Por qué importa?
- ¿Qué campo oculto FALTA en todos los formularios POST de la aplicación?

3.2 Explotación

El archivo attacks/csrf-attack.html simula un sitio externo malicioso. Debes demostrar que, mientras una víctima tiene sesión activa en `http://localhost:8000`, abrir este archivo en una pestaña nueva del mismo navegador elimina una de sus notas sin su consentimiento.

Pasos del ataque

- Inicia sesión como `estudiante@uatf.edu / 123456`. Verifica que tienes al menos 2 notas.
- Anota el ID numérico de la nota que quieres eliminar (inspecciona el formulario de eliminación en la lista).
- Abre `attacks/csrf-attack.html` en una pestaña NUEVA del mismo navegador. Puedes abrirlo como `file://` o servirlo desde otro puerto.
- **Ajusta el target_id vía URL, por ejemplo:** `file:///ruta/al/archivo.html?target_id=4`
- Vuelve a `http://localhost:8000/notes/index.php` y confirma que la nota fue eliminada.

Capturas requeridas (CAP-3.A y CAP-3.B)

- CAP-3.A: Lista de notas ANTES del ataque, con el ID objetivo visible.
- CAP-3.B: Lista de notas DESPUÉS del ataque mostrando que el ID objetivo desapareció.

3.3 Corrección

Implementa tres medidas de defensa en profundidad:

Defensa 1: Flags de cookie seguros

En config.php, ANTES de `session_start()`, activa:

```
ini_set('session.cookie_httponly', 1);
```



```
ini_set('session.cookie_samesite', 'Lax');
ini_set('session.use_strict_mode', 1);
```

Defensa 2: Regeneración del ID de sesión

En login.php, inmediatamente DESPUÉS de validar las credenciales y ANTES de asignar \$_SESSION['user_id'], invoca session_regenerate_id(true) para mitigar la fijación de sesión.

Defensa 3: Token CSRF en todos los formularios POST

Define dos helpers en config.php:

```
function csrf_token() {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function csrf_check() {
    $token = $_POST['csrf_token'] ?? '';
    if (empty($_SESSION['csrf_token']) ||
        !hash_equals($_SESSION['csrf_token'], $token)) {
        http_response_code(419);
        exit('Token CSRF inválido.');
```

Agrega un input oculto con el token en cada formulario POST, y llama csrf_check() al inicio de los endpoints store.php, update.php, delete.php, login.php.

Captura requerida (CAP-3.C)

- CAP-3.C: Después del parche, reabrir attacks/csrf-attack.html con sesión activa y mostrar que la aplicación devuelve "Token CSRF inválido" (HTTP 419) y que la nota NO fue eliminada.

Desglose de 25 puntos

Criterio	Pts
Identificación de los tres problemas (flags, regenerate_id, token faltante)	5
Ataque CSRF ejecutado exitosamente (CAP-3.A + CAP-3.B)	7
Defensa 1: flags de cookie httponly/samesite/strict_mode	3
Defensa 2: session_regenerate_id(true) tras login	3
Defensa 3: token CSRF con hash_equals en todos los POST	5
Verificación del parche (CAP-3.C)	2

3. Informe Final (10 puntos)

Elabora un documento breve (máximo 3 páginas) con la siguiente estructura:

Portada

- Nombre completo, carnet, fecha, materia.

Una sección por cada una de las 3 secciones del examen

- Líneas de código vulnerables identificadas (nombre de archivo + número de línea).
- Payload o mecanismo de explotación utilizado.
- Capturas de pantalla requeridas, numeradas según el código CAP-X.Y.
- Fragmento de código del parche aplicado (diff o bloque completo).

Reflexión breve (1 párrafo, mín. 100 palabras)

Respondiendo: ¿Qué vulnerabilidad te pareció más peligrosa para un sistema real? ¿Qué protección automática de Laravel (de las vistas en las guías 1-5) hubiera bloqueado el ataque sin que el desarrollador escribiera ni una línea extra?

4. Entregables y Formato

Antes de salir del laboratorio

1. Sube el código corregido como ZIP con tu nombre: examen-INF781-APELLIDO-NOMBRE.zip
2. Sube el informe en PDF: informe-INF781-APELLIDO-NOMBRE.pdf
3. Ambos archivos se entregan en el aula virtual del curso antes de que acabe el tiempo.
4. No se aceptan entregas posteriores al cierre del examen.

Rúbrica consolidada

Sección	Puntos
Sección 1 — SQL Injection	30
Sección 2 — XSS + Hashing MD5	30
Sección 3 — CSRF + Sesiones	25
Informe final	10
Calidad general del código parcheado (sin warnings, sin código muerto)	5
TOTAL	100

Buenas prácticas de tiempo

- 10 min setup + verificación → empezar cronómetro oficial.
- 20 min Sección 1 (SQLi): es la más mecánica, resuélvela primero para sumar confianza.
- 25 min Sección 2 (XSS + MD5): la parte B se puede hacer en paralelo mientras renderiza la A.
- 25 min Sección 3 (CSRF + Sesiones): el ataque es rápido, el parche requiere cuidado con los tokens.
- 10 min informe + empaquetado ZIP + subida al aula virtual.

Honestidad académica

Durante el examen no está permitido consultar internet, los repositorios de las prácticas anteriores ni comunicarte con compañeros. Puedes consultar las guías 1 a 4 en papel o PDF local si las tienes contigo. Cualquier intento de deshonestidad anula el examen.